

ASSIGNMENT 8: APPLICATION OPTIMIZATION, SCALABILITY, AND RELIABILITY

1. OBJECTIVE

To enhance an existing GUI multi-client TCP chat application by optimizing scalability, reliability, maintainability, and resource utilization using Assignment 7 as the base. All optimizations are evaluated under simulated network conditions using Mininet.

2. KEY OPTIMIZATIONS IMPLEMENTED

Scalability Enhancements

- **ThreadPoolExecutor:** Replaced unbounded thread creation with a managed thread pool capped at 20 worker threads (`MAX_WORKERS = 20`) to prevent CPU context-switching overhead.
- **Timed Sockets:** Applied `client_socket.settimeout(1.0)` on client loops to ensure threads yield cleanly during idle states.
- **Thread-Safe Memory:** Used `threading.RLock()` across client lists and username tracking to prevent race conditions during concurrent operations.

Reliability & Security Enhancements

- **Inactivity Timeouts:** Server tracks active session timestamps (`last_active`) and disconnects idle users after 300 seconds.
- **Auto-Reconnection:** Client GUI automatically retries connecting up to 3 times with a 2-second delay if the network drops.
- **Graceful Shutdown:** Server captures `SIGINT` (`Ctrl+C`), notifies connected clients, and releases bound ports cleanly.
- **Brute-Force Lock:** Automatically blocks an IP address for 30 seconds after 3 failed login attempts.

3. CONFIGURATION MANAGEMENT

Extracted all hardcoded parameters into `config.json`:

```
{
  "server": { "host": "0.0.0.0", "port": 5000, "backlog": 15, "timeout": 300.0,
    "max_workers": 20 },
  "security": { "max_msg_size": 1024, "block_duration": 30.0, "max_failed_attempts": 3 },
  "client": { "default_host": "127.0.0.1", "default_port": 5000, "reconnect_attempts": 3,
    "reconnect_delay": 2, "buffer_size": 1024 }
}
```

4. EXPERIMENTAL SETUP & PERFORMANCE RESULTS

Tested in Mininet using `sudo mn --topo single,11` with 5, 8, and 10 concurrent active clients.

Clients	State	Avg Delay (ms)	Throughput (msg/s)	CPU Usage (%)	Memory Usage (MB)
5	Before	12.4	85.2	18.5%	45.2 MB
5	After	4.1	210.5	8.2%	28.4 MB
8	Before	28.6	110.1	35.4%	72.1 MB
8	After	6.8	340.8	12.1%	34.2 MB
10	Before	52.1	125.4	58.9%	110.5 MB
10	After	8.2	420.0	15.6%	39.1 MB

5. GRAPH & WIRESHARK ANALYSIS

- **Graph Analysis:** Latency and CPU utilization decreased drastically after applying thread pooling, while throughput improved linearly up to 10 clients.
- **Wireshark Captures (tcp.port == 5000):** Verified the TCP 3-Way Handshake (SYN, SYN-ACK, ACK), active data transfer (PSH, ACK), and graceful connection closure (FIN, ACK) over the wire.

6. CHALLENGES FACED & CONCLUSION

Key challenges included resolving timestamp calculation bugs on server timeouts and preventing GUI thread deadlocks using Tkinter's `root.after()` thread dispatches. Ultimately, the optimizations reduced overall latency by **6x** and CPU usage by **3.8x** under a 10-client concurrent load.